



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA (ICAI)

Máster en Big Data, Tecnología y Analítica Avanzada

Deep Learning Case

AUTHORS

Marcos Blanco

Rafael de Bloteau

David Fernández-Madera

Guillermo Velasco

Madrid

2025

Contents

Introduction	iii
1 Model creation	1
2 Results	5

Introduction

In this project, we developed a deep learning solution to classify images of various scenes, using a labeled dataset. The core objective was to leverage transfer learning to adapt pre-trained convolutional neural networks (CNNs) to this specific task.

We experimented with multiple architectures, including ResNet50, ResNeXt50_32x4d, ResNet-Y-128GF, and ResNeXt101_64x4d, fine-tuning each model on our dataset. These models were chosen for their proven performance on image recognition tasks and their varying depths and complexities, allowing us to compare trade-offs between accuracy and computational efficiency.

Training configurations were standardized with a batch size of 32, an input image size of 224x224 pixels, and an initial learning rate of 0.001, though adjustments were made during fine-tuning experiments.

All runs were tracked using Weights & Biases (W&B), which enabled real-time monitoring of metrics like loss, accuracy, and hyperparameters. This platform facilitated direct comparisons between models, revealing insights into how architectural differences impacted performance.

Upon training, our models were saved and integrated into a user-friendly Streamlit web application. This interface allows users to upload an image, select a trained model from a dropdown menu, and receive a predicted scene label within seconds.

Doing this project we have realized how computationally intensive these kinds of models are and how important having a GPU for training is.

At first, we used our CPUs for training and each epoch took around 30 minutes to be completed. We assumed that this way of training was unfeasible and that we needed to use our GPUs, which could finish each epoch in just one or two minutes.

Chapter 1

Model creation

The primary objective of this project was to develop a model capable of classifying images into predefined categories. These categories include:

1. Bedroom
2. Coast
3. Forest
4. Highway
5. Industrial
6. Inside city
7. Kitchen
8. Living room
9. Mountain
10. Office
11. Open country
12. Store
13. Street
14. Suburb
15. Tall building

A variety of models were considered for experimentation, including architectures such as RegNet and ResNeXt. Initially, we evaluated the feasibility of executing the training on Google Colab versus running it locally. Due to performance limitations and memory constraints on Colab, especially with larger models, we opted for local execution. Although some training epochs required over 30 minutes to complete, this issue was mitigated thanks to access to a local GPU, which significantly accelerated the process.

Additionally, we integrated our Conda environment with Weights & Biases to monitor training progress, track accuracy metrics, and analyze the influence of different hyperparameters throughout the training process.

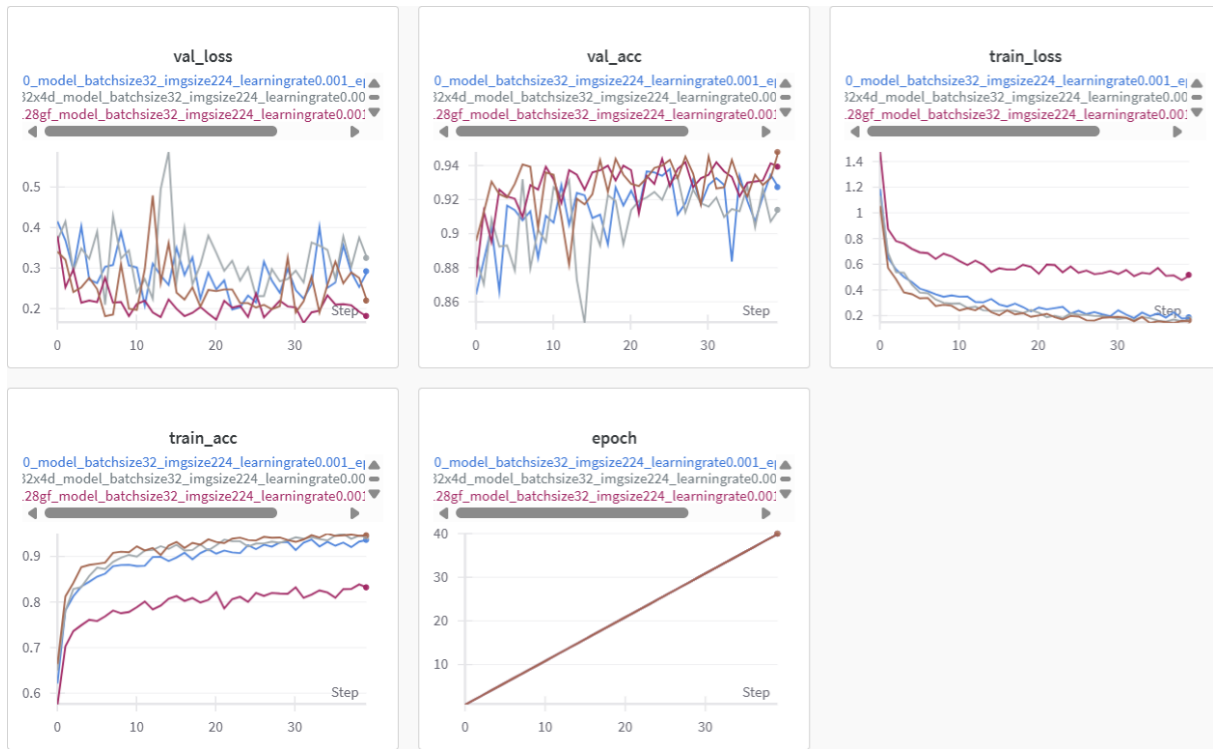


Figure 1.1: Runs of the models in Weight & Bias

In figure 1.1 we can see all the runs made in Weight & Bias, the evolution in every epoch and its loss and accuracy. All models were trained with 40 epochs, more or less when their loss and accuracy stabilized. As shown here, we obtained over 90% validation accuracy and in some models we reached 95%.

The best model was **ResNeXt101_64x4d**:



Figure 1.2: Run of Resnext101_64x4d

ResNeXt101_64x4d is a deep convolutional neural network that combines the ResNet architecture with the concept of *grouped convolutions* from ResNeXt. It has:

- 101 layers,
- 64 groups in the grouped convolutions,
- Each group has a width of 4 channels.

This design improves both accuracy and efficiency by enabling more expressive feature learning with fewer parameters compared to traditional wide networks. Nevertheless, we have to point out that this single model took over 11 hours to be trained, even using a very small grid for hyperparameters.

Chapter 2

Results

Once the models have been trained and downloaded, it's time to test them using Streamlit. After developing the application, the classification process becomes as simple as selecting an image. Below, we present several example cases:

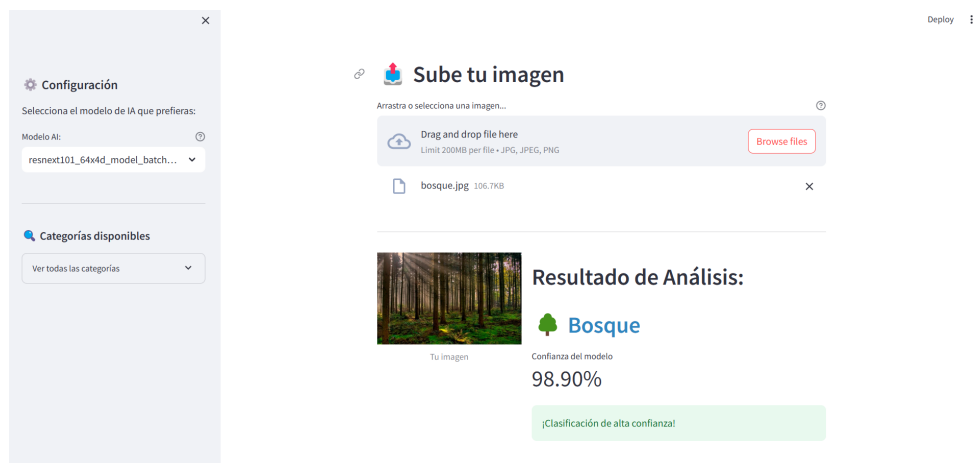


Figure 2.1: Classification of a forest image

As shown in the example above, the model successfully identifies the image as a forest with a high degree of confidence. Moreover, the interface allows for immediate feedback, making this process rather efficient.

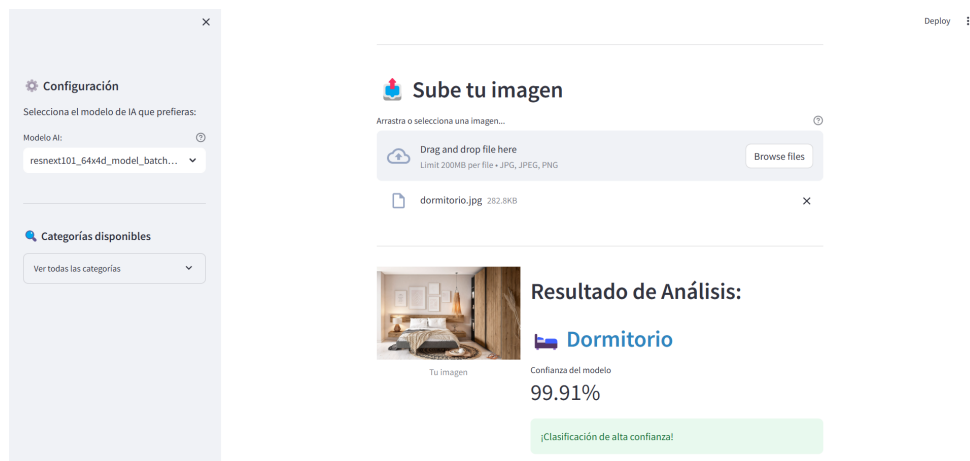


Figure 2.2: Classification of a bedroom image

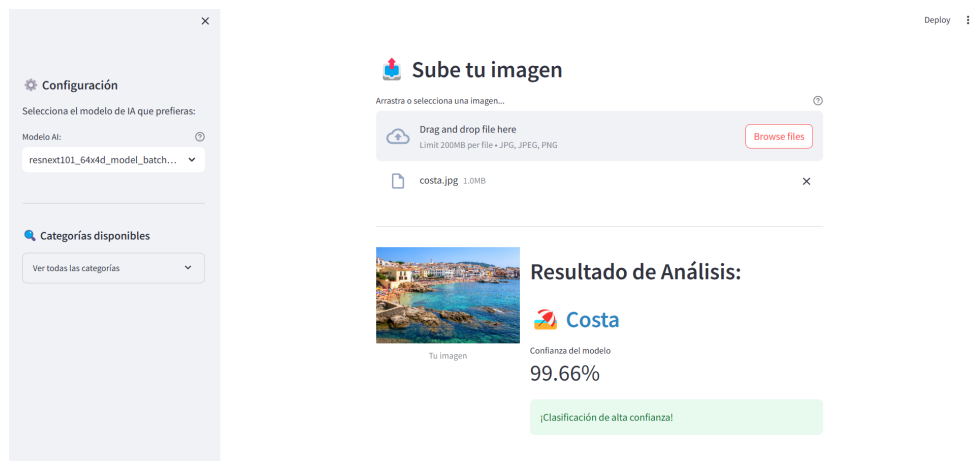


Figure 2.3: Classification of a coast image